

NeurIPS 2022 | 让模型在自己的错误上再训练一次：自动驾驶开环/闭环鸿沟的一个简单解法

Woven Planet 提出 CW-ERM：先在仿真里跑一遍策略，找出翻车的场景，把它们上采样后重新训练，某些指标上碰撞最多减少约 35%，且无需可微仿真器。

如今大多数自动驾驶规划器都用行为克隆（behavioral cloning）来训练：把人类专家的驾驶记录喂给网络，让它去复现专家的下一步动作。训练是开环（open-loop）进行的，一步一步地在离线数据上对齐专家动作。但当策略真正上路时，它是闭环（closed-loop）运行的：它做出的每一个动作都会改变自己下一刻看到的状态。一次微小的转向误差，会把车推到一个专家从未到过的、略微偏移的位置，下一步预测就更差一点，误差不断累积。这就是协变量漂移（covariate shift），也是为什么一个开环指标看起来很漂亮的规划器，仍然可能在路上一路漂移直到发生碰撞。

还有第二个更隐蔽的问题。人们真正在乎的指标，比如有没有碰撞、有没有压线，都是不可微的。训练损失无法直接看到它们，于是开环训练和闭环部署实际上在优化两个不同的目标。一个直接的补救办法，是把仿真器塞进训练回路里、对它做反向传播，像 Urban Driver 这类方法那样。它确实有效，但代价高昂：需要一个可微仿真器，扩展性差，训练时展开策略还非常吃显存。

这篇来自 Woven Planet 的 NeurIPS 2022 论文提出了一个更简单的问题：如果仿真器只用来决定哪些训练场景更重要、而完全不碰损失函数，会怎样？他们的答案就是 CW-ERM（Closed-loop Weighted Empirical Risk Minimization，闭环加权经验风险最小化），一个两步走的配方：找出策略会失败的场景，然后在训练最终策略时，恰恰把这些场景上采样。整套方法都浓缩在下面这张图里。

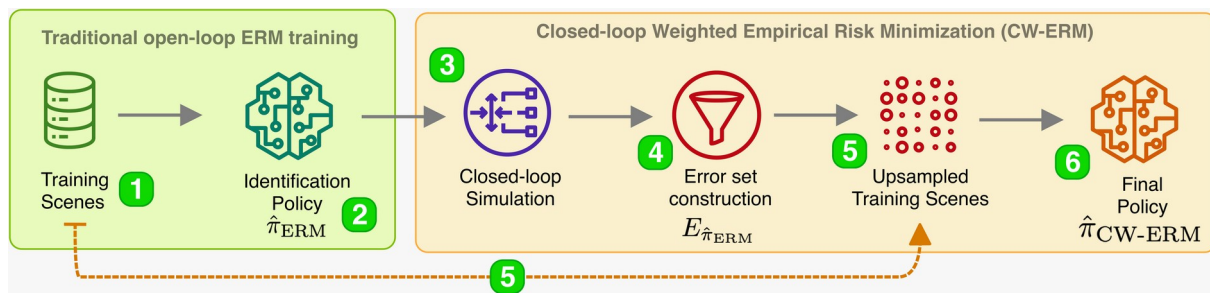


图 1 CW-ERM 方法总览。步骤 1-2：用普通 ERM 训练一个识别策略。步骤 3-4：在闭环仿真器中回放它，收集代价指标（如碰撞）来构造错误集。步骤 5-6：把错误集中的场景上采样，在加权后的数据上重新训练最终策略。

论文：<https://arxiv.org/abs/2210.02174> · 代码：<https://github.com/wp-research-uk/cw-erm>

为什么开环训练会悄悄拖累闭环驾驶

经验风险最小化（Empirical Risk Minimization，ERM）是标准的行为克隆目标：在专家自身分布采样出的状态上，最小化逐步模仿损失的期望。策略只会被问到专家到访过的状态，因此它从来没有学过如何从自己的错误中恢复。而在部署时分布翻转了，状态现在来自策略自己的滚动轨迹，这两个分布之间的错配，正是让车变得不安全的根源。由于碰撞不可微，ERM 损失并不能给出任何推动策略去规避碰撞的梯度。作者的洞见是：你并不需要一个穿过指标的梯度；你只需要知道策略在哪些场景上出错，而这一点，用一个普通的、不可微的仿真器就能拿到。

方法：三个阶段，一个加权项

CW-ERM 刻意做得很简单，可以拆成三个阶段。第一，用普通方式训练一个识别策略，也就是标准的 ERM 行为克隆。第二，把这个策略放进闭环仿真器，在训练场景上滚动一遍，记录一个代价，比如碰撞次数；每一个代价为正的場景，也就是策略失败的場景，都进入错误集。第三，再次用加权 ERM 训练最终策略，此时错误集里的场景被按因子 w 上采样。最终的目标函数几乎和普通行为克隆一模一样，只多了一个完全由闭环失败驱动的加权项。

有两个实践细节让它真正奏效。识别策略被刻意提前停止，只训练 K 个 epoch，这样它仍然会足够频繁地失败，从而填满一个有意义的错误集；如果训练到收敛，错误集就被掏空了。另外，作者并不是去重新加权损失，而是按一个整数因子把失败场景真正复制上采样，他们发现这样更稳定。因为仿真器只用来给场景打标签、从不用来计算梯度，所以任何不可微指标都能驱动这个加权，也完全不需要可微仿真器。论文还把这种“按失败重加权”的机制，与经典的、通过密度比估计来纠正协变量漂移的思路联系起来，为这个配方给出了一个有原理支撑的骨架。

结果：各项碰撞指标全面下降

方法在一个专有的真实城市驾驶数据集上评估，这些数据由 Woven Planet 的自动驾驶车队在 San Francisco 和 Palo Alto 的高难度任务中采集，包含自车与周围智能体的记录轨迹以及高精地图。场景既难又多样，包括跟停前车、在路口停住、在有行人和骑行者的密集车流中穿行，每段大多持续 11 到 13 秒，最长可达 30 秒。训练用了 180 小时的驾驶数据，另有各 60 小时分别留作验证与测试，最终策略训练 40 个 epoch。闭环仿真器与相关指标已经开源发布。

与最强的基线，也就是带专家扰动的 ERM 行为克隆相比，CW-ERM 在每一项指标上都显著减少了碰撞，其中某些指标的改进达到约 35%。当模型对前向碰撞场景做上采样时，前向碰撞从 14 次降到 9 次，约减少 36%。当它对侧向碰撞场景做上采样时，侧向碰撞从 55 次降到 47 次。这里还有一个可喜的副作用：改进一个指标常常会顺带改进相关指标。对侧向碰撞上采样也降低了后向碰撞，这说明策略是在变得不那么被动谨慎，而不是在钻单一数字的空子。在若干情形下方差也比基线更低，因此这些收益是稳定的，而非侥幸。图 2 展示了各指标、各 CW-ERM 变体下的完整对比。

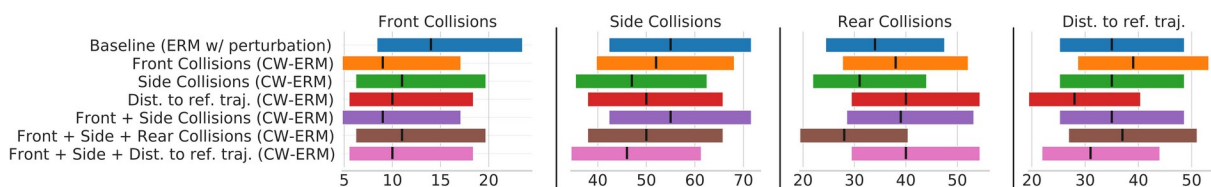


图 2 四项碰撞指标（前向、侧向、后向、到参考轨迹的距离）下的闭环评估。每一行是一个上采样不同指标的 CW-ERM 变体；最上面一行是最强基线（带扰动的 ERM）。误差条为来自精确二项后验（flat prior）的 0.95 置信区间。针对某个指标会压低该指标，而改进往往会外溢到相关指标。

定性行为和总体数字一样说明问题。图 3 展示了一个测试场景：基线驶入路口，无视了从右侧驶来的车辆，造成一次前向碰撞；而用前向碰撞场景上采样训练出来的 CW-ERM 策略，则会在路口等待，直到路径清空。这个变化并不是轨迹上的表面微调，而是策略在它过去恰恰会失败的那类情形里，学会了让行。

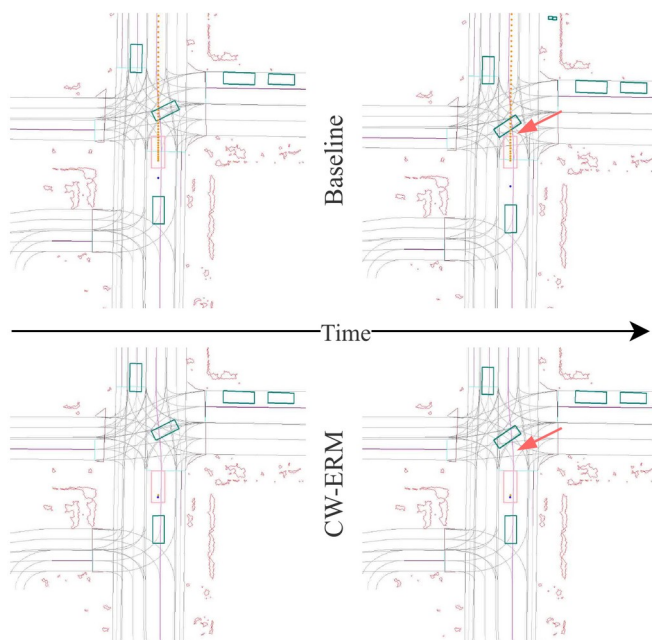


图 3 一个测试场景，上为基线、下为前向碰撞上采样的 CW-ERM，从左到右是时间推进。蓝点是目标轨迹，黄点是预测轨迹；自车是粉色框，其他智能体是青色框。基线驶入路口，与从右侧来的车相撞（红色箭头）；CW-ERM 则选择等待。

该上采样多少，又该组合哪些指标

有两个旋钮决定这个配方效果好不好。第一个是上采样因子 w 。图 4 在验证集上把它对侧向碰撞做了扫描：性能稳步改善，直到约 $w = 50$ ，此后侧向碰撞又开始回升，这与 Just Train Twice (JTT) 方法在其上采样方案中报告的饱和现象如出一辙。第二个旋钮是识别策略的提前停止预算 K ：单指标错误集大约 10 个 epoch 最好，多指标错误集则是 20 个。二者合在一起，给出的是一个小而清晰的搜索空间，而不是一个脆弱的空间。

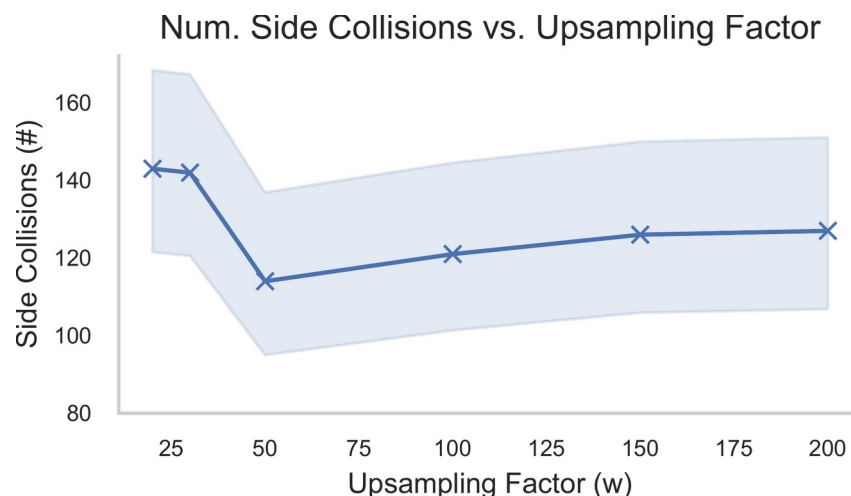


图 4 验证集上侧向碰撞随上采样因子 w 的变化。性能在约 $w = 50$ 之前持续改善，超过之后开始退化；阴影带是置信区间。

此外还有一个选择：该针对哪些失败。对单个指标上采样能在该指标上带来最大收益，而组合多个指标则会沿着帕累托（Pareto）前沿进行权衡，因此具体怎么选，取决于某个部署最在乎的是哪一种失败。把前向、侧向和到参考轨迹的距离组合起来效果不错，但一旦加进后向碰撞就会出现明显的回退。作者把原因追溯到后向碰撞指标里的假阳性：仿真器是从日志回放其他智能体的，因此一辆现实中本会刹车的回放车会径直撞上自车，却被记成自车的过错。这里的实践教训是：方法的可靠程度，取决于它上采样的那个指标有多可靠，一个有噪声的指标反而会把训练带偏。与此同时，到参考轨迹的距离失败相对最强基线从 35 降到 28，约减少 20%，而且这种重加权完全不触碰推理阶段，因此在行车时不会带来任何额外开销。图 5 给出了侧向碰撞这一维度上一个对应的定性挽救：基线偏离了目标轨迹、撞上前方的公交车，而 CW-ERM 策略则在公交车后减速，避免了碰撞。

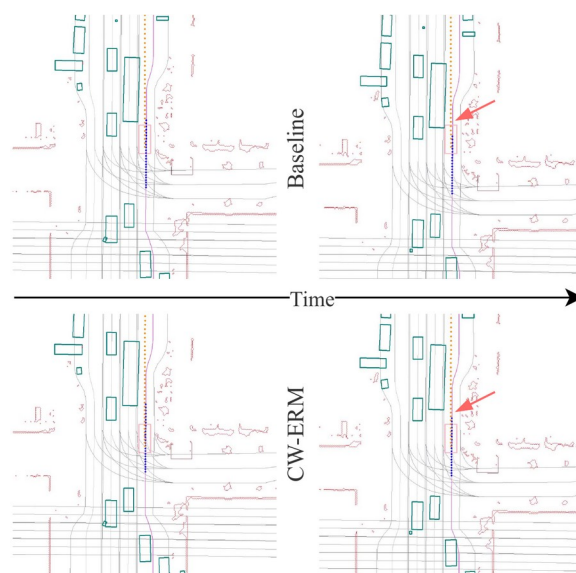


图 5 上为基线、下为侧向碰撞上采样的 CW-ERM，随时间推进。基线偏离目标轨迹，与前方公交车相撞（红色箭头）；CW-ERM 在公交车后减速，避免了碰撞。

一张紧凑的成绩单

针对最强基线的主要对比可以一目了然地总结出来。表中每个 CW-ERM 数字都来自上采样对应指标的那个变体。

表 1 CW-ERM 对比最强基线（带扰动的 ERM）在测试集上的结果，越低越好。

指标	基线	CW-ERM	变化
前向碰撞	14	9	约减少 36%
侧向碰撞	55	47	减少
到参考轨迹距离的失败	35	28	约减少 20%

为什么它重要

CW-ERM 的吸引力在于它索取得如此之少。你不需要可微仿真器、不需要人在回路，也不需要昂贵的闭环训练，就能拿到闭环带来的好处。你只要在仿真里把策略跑一遍，记下它在哪里失败，把那些场景上采样，然后重新训练。这个配方带来了实实在在的碰撞下降，适用于任何闭环指标，而且不增加任何推理延迟，因为所有额外的开销都发生在训练阶段。诚实的边界也值得一提：这些收益是在某一家公司的专有城市数据集上测得的；后向碰撞的结果暴露出方法对指标质量有多敏感；仿真器里从日志回放的智能体，也可能制造出并不真正归咎于自车的失败。即便如此，作为一种便宜、与指标无关、又有理论支撑的手段，去把一个模仿学习出来的规划器往更安全的闭环行为上推一把，CW-ERM 依然是一个真正实用、团队可以低成本上手的想法。